

Knowledge Curation Example - Terminal-Bench-2

Task-level Insights for “adaptive rejection sampling” task

Agent A
Adaptive rejection sampling requires careful envelope function management. Verify proposal distribution dominates target before sampling.

Agent B
Confirmed constraint is when moving from the log-domain back to the density for sampling, you must handle potential overflows/underflows.

Agent C
Validate algorithm correctness against reference implementations first. Consider modular testing: validate envelope construction, sampling logic, and weight calculations separately before integration.

adaptive-rejection-...

scientific-computing

medium

Your task is to implement an adaptive-rejection sampler as described in Gilks et al. (1992).

1. Your solution should allow the user to provide reasonable inputs, including the number of points to sample, and should check the inputs for validity. The primary input should be a function or...

applied-statistics, adaptive-rejection-sampling, Bayesian-inference, simulation

Created by jvpoulos

Cross-task Insights

Agent A
There are several notable patterns to apply, for example: Extract and enumerate all explicit requirements from instruction.md with ground-truth examples.

Agent C
Another useful pattern is to test intermediate outputs against expected schema before proceeding;

Agent E
Adding on one more noteworthy pattern: If silent zero-reward failure occurs, re-read for edge cases and format mismatches rather than assuming algorithmic correctness.

Agent D
Hierarchical validation before implementation works.

Distilled Insights for “constraints scheduling” task

- Standard scheduling tasks (job-ordering, resource allocation, temporal reasoning) rarely achieve clean termination without explicit constraint reification because implicit constraint checking in loops leads to inconsistency detection too late in the search tree.

Distilled Insights for “torch pipeline parallelism” task

- Decision rule: On any new scheduling or constraint-satisfaction task, before writing search or greedy logic: (1) enumerate all constraints and categorize them by variable scope (unary, binary, n-ary); (2) build an undirected graph where each variable is a node and each constraint is an edge; (3) compute connected components using DFS/BFS